Course: CS6650

Assignment: Homework 7 — When Your Startup's Flash Sale Almost Failed

Team Members: Rahul CJ, Wilson Neira, Supriya, Yalin Sun

Individual Contributions

Rahul CJ: Built and deployed the full infrastructure (VPC, ALB, ECS, SNS, SQS, Lambda) using Terraform in us-west-2. Implemented the Go service with sync/async endpoints and configurable goroutine-based processor. Ran all Locust load tests (sync normal, sync flash, async flash) and worker scaling experiments (1, 5, 20, 100 goroutines). Deployed Lambda with SNS trigger and captured cold/warm start logs from CloudWatch. Performed cost analysis and wrote architecture insights.

Wilson Neira: Built and deployed the full infrastructure using Terraform in us-east-1. Implemented the Go order system with buffered-channel payment gate, sync/async endpoints, and SQS processor with configurable workers. Ran all Locust tests and worker scaling experiments (1, 5, 20, 100 goroutines). Deployed Lambda function, captured cold/warm start CloudWatch logs, and performed cost analysis. Documented deployment steps with code screenshots.

Supriya: Built and deployed the full infrastructure using Terraform. Implemented the Go service with sync and async endpoints using buffered-channel bottleneck simulation. Ran all Locust load tests and worker scaling experiments. Deployed Lambda with SNS trigger and observed cold start behavior. Performed cost and trade-off analysis. Wrote detailed architecture overview and analysis answers.

Yalin Sun: Built and deployed the async order-processing architecture using Terraform, including SNS, SQS, and ECS services. Implemented Go services with sync and async endpoints and simulated the payment bottleneck with buffered channels. Conducted Locust load tests and worker scaling experiments to analyze system behavior under flash-sale traffic. Implemented a serverless alternative by deploying a Lambda function triggered by SNS and observed cold and warm start behavior through CloudWatch logs. Completed cost comparison and architecture trade-off analysis between ECS workers and Lambda.

Part II: Synchronous vs Async

Architecture

Sync path: Client → ALB → Order API → Payment (3s) → Response

Async path: Client → ALB → Order API → SNS → SQS → Processor → Payment (3s)
All three team members implemented the same architecture using Terraform-provisioned infrastructure on AWS with ECS Fargate, SNS, and SQS. The payment bottleneck was simulated using a buffered channel (not just time.Sleep) to enforce real throughput limiting at 0.33 orders/second.

Phase 1: Sync Results — Team Comparison

Normal Load (5 users, 30s, spawn rate 1/s)

Metric	Rahul CJ	Wilson Neira	Supriya	Yalin
Total Requests	14	9	9	40
Failures	0	0	0	0
Median Response Time	14,000 ms	12,000 ms	10,405 ms	3000 ms
Avg Response Time	12,049 ms	10,528 ms	10,405 ms	3000 ms
p95 Response Time	15,000 ms	15,000 ms	14,763 ms	3000 ms
RPS	~0.33	~0.4	~0.33	~1.6

All three members observed that even normal load (5 users) causes significant queuing because all concurrent users compete for the single 0.33 orders/s payment gate. Response times climb to 10–15 seconds despite 0% failure rate.

Yalin got a slightly different results as she used a buffered channel with capacity 15 to simulate a realistic payment bottleneck. In her scenario, Under normal operations, response times were highly consistent because each request performed the same synchronous payment verification with an approximately fixed 3-second delay. Since the system was not yet saturated, there was little queueing, so the median, 95th percentile, and 99th percentile were all very close to each other.

Flash Sale (20 users, 60s, spawn rate 10/s)

Metric	Rahul CJ	Wilson Neira	Supriya	Yalin Sun
Total Requests	18	19	19	285
Failures	0	0	0	0
Median Response Time	27,000 ms	30,000 ms	29,550 ms	3300 ms
Avg Response Time	28,341 ms	29,694 ms	29,550 ms	3666 ms
Max Response Time	53,000 ms	56,181 ms	56,036 ms	5035 ms
RPS	~0.33	~0.4	~0.33	~5.2

Under flash sale load, all three systems showed the same pattern: only 18–19 requests completed in 60 seconds, with response times ballooning to 30–56 seconds. The system never returned HTTP errors, but the user experience was completely broken — customers would wait nearly a minute for confirmation.

Yalin's experiment handled a higher request load, with about 285 requests completed in 60 seconds. The response time also increased, and no request failures were observed.

Phase 2: Bottleneck Analysis

All three members arrived at the same math:

- Payment gate: 1 order / 3 seconds = 0.33 orders/second
- Max throughput with 20 concurrent users: ~0.33 orders/s (the payment gate is the bottleneck, not CPU/memory/network)
- Flash sale demand (from assignment spec): 20 orders/second
- Orders lost per second: 20 − 0.33 = 19.67 orders/s

The bottleneck is the sequential nature of synchronous payment processing. The only fix is to decouple order acceptance from payment execution.

Yalin got a different result. In her synchronous design, each payment verification takes 3 seconds. A buffered channel with capacity 15 is used, which means at most 15 payment verifications can run at the same time. Therefore, the theoretical maximum throughput of the payment processor is 15 / 3 = 5 orders per second. Orders lost per second is 20 - 5 = 15.

Phase 3: Async Results — Team Comparison

Flash Sale Async (20 users, 60s, spawn rate 10/s)

Metric	Rahul CJ	Wilson Neira	Supriya	Yalin Sun
Total Requests	3,853	3,540	3,804	3451
Failures	0	0	0	0
Median Response Time	93 ms	34 ms	12 ms	40 ms
Avg Response Time	99 ms	38 ms	14 ms	86 ms
p95 Response Time	140 ms	55 ms	—	60 ms
RPS	~49.8	~57.5	~63.53	~57.7

All systems achieved 100% acceptance rate with sub-100ms response times. The async endpoint accepted roughly 3,400–3,850 orders in the same time the sync endpoint handled only 18–19 — a roughly 186–214x improvement. Response times dropped from ~30 seconds to under 100 milliseconds. The variation in RPS across team members likely reflects differences in network latency to the ALB and regional deployment (us-west-2 vs us-east-1).

Phase 4: Queue Behavior

Metric	Rahul CJ	Wilson Neira	Supriya	Yalin Sun
Acceptance Rate	~49.8/s	~59/s	~63.53/s	~60/s
Single Worker Rate	0.33/s	0.33/s	0.33/s	0.33/s
Queue Growth Rate	~49.47 msgs/s	~58.67 msgs/s	~63.2 msgs/s	~59.67 msgs/s
Peak Queue Depth	~7,400	~3,430	~948 (initial)	~4000
Drain Time (1 worker)	~20 min observed	~176 min calculated	~190 min calculated	~202 min calculated

All members observed the same fundamental problem: the queue grows rapidly because orders are accepted far faster than they are processed. The differences in peak depth reflect how many test runs were executed (accumulated messages) and whether the queue was cleared between runs.

Phase 5: Worker Scaling — Team Comparison

Processing rate formula: workers × (1/3) orders/s

Workers	Rate (orders/s)	Rahul CJ Drain	Wilson Neira Drain	Supriya Drain	Yalin Sun
1	0.33	~6.2 hours (theoretical)	~181 min (calculated)	~190 min (calculated)	~202 min (calculated)
5	1.67	~74 min	—	—	~33 min (calculated)
20	6.67	~18.5 min	—	—	~5.3 min (calculated)
100	33.33	~3.7 min	—	—	~30s

All members confirmed that scaling goroutines within a single ECS task increases processing throughput linearly. The Locust acceptance rate remained consistent (~3,500– 3,850 requests per 60s test) regardless of worker count since the async endpoint performance is independent of backend processing speed.

Minimum workers to prevent queue buildup at 60 orders/s: 60 × 3 = **180 goroutines**. All members arrived at the same answer. As Wilson noted, whether 180 goroutines is practical in a single ECS task with 256 CPU / 512MB memory is questionable — in production, horizontal scaling across multiple tasks would be necessary.

Part II Analysis Questions

How many times more orders did async accept compared to sync?

Member	Async Requests	Sync Requests	Multiplier
Rahul CJ	3,853	18	~214x
Wilson Neira	3,540	19	~186x
Supriya	3,804	19	~200x
Yalin Sun	3451	285	~12x

Average across team: approximately **200x** more orders accepted via async.

What causes queue buildup and how do you prevent it?

Queue buildup occurs when the order acceptance rate exceeds the processing rate. All team members identified the same prevention strategies: increase worker count (more goroutines or more ECS tasks), autoscale based on CloudWatch queue depth metrics (ApproximateNumberOfMessagesVisible), reduce per-order processing cost, or apply backpressure/rate limiting on the input side.

When would you choose sync vs async in production?

The team consensus: sync is appropriate when the client must know the final result immediately and processing is fast enough (e.g., balance checks, inventory lookups, real-time fraud decisions). Async is appropriate for slow or bursty workloads where fast acknowledgment matters more than immediate completion, and where the system needs to absorb traffic spikes without degrading user experience.

Part III: Serverless (SNS → Lambda)

Architecture Change

All three members replaced the SNS → SQS → ECS Worker pipeline with SNS → Lambda. The same SNS topic from Part II was reused, the SQS subscription was removed, and a Lambda function subscribed directly to SNS.

Lambda configuration: provided.al2 runtime (Go), 512MB memory, same 3-second payment delay, SNS trigger.

Cold Start Observations — Team Comparison

Metric	Rahul CJ	Wilson Neira	Supriya	Yalin Sun
Cold Start Init Duration	69.57 ms	73 ms	63.19 ms	71.76 ms
Total Duration (cold)	3,005.51 ms	~3,003 ms	3,004.49 ms	3002.12 ms

---

Warm Duration	3,001.67 ms	3,001.61 ms	3,001.55 ms	3003.58 ms
Cold Start Overhead	2.31%	~2.4%	2.1%	2.4%
Memory Used	20 MB	20 MB	20 MB	20 MB

All members observed consistent behavior: cold starts occurred on the first invocation and after periods of inactivity (~5–15 minutes idle). The init duration ranged from 63–73ms across the team, representing only 2.1–2.4% overhead on the 3-second payment processing task — negligible for this workload.

Cost Analysis

- ECS baseline: 2 tasks × $8.50/month = **$17/month** (always running)
- Lambda (10,000 orders/month): 10,000 requests + 15,000 GB-seconds = **$0** (within free tier)
- Lambda break-even: free tier covers ~267K orders/month; need ~1.7M requests/month to reach $17

Trade-off Analysis

What You Gain	What You Lose
Zero operational overhead	No SQS durability guarantees
Pay only for what you use	SNS retries twice then discards
Automatic scaling to any load	No batch processing capabilities
No queue/worker management	Cold start delays (~63–73ms)

Team Recommendation

All members recommend switching to Lambda for this startup at its current stage. The reasoning is consistent across the team:

- The cold start overhead (2.1–2.4%) is negligible on a 3-second workload
- Cost is effectively $0 at low volume vs $17/month for ECS
- Eliminating queue management, worker scaling, and ECS health monitoring significantly reduces operational burden

The key caveat all members identified: if order durability and retry control are critical (i.e., losing an order because SNS retried twice and gave up is unacceptable), the SNS → SQS → ECS path is safer. A reasonable middle ground would be adding a dead letter queue to the Lambda subscription to capture failed messages. At 267K+ orders/month, the cost equation shifts back in favor of ECS.

Conclusion / Architecture Insights

This assignment demonstrated a clear progression through three architectural patterns for handling load:

1. **Synchronous processing** failed to scale because the payment gate capped throughput at 0.33 orders/s regardless of how many users connected. All three team members observed identical behavior: 18–19 requests in 60 seconds with response times approaching a minute.

2. **Async processing with SNS/SQS** solved the acceptance problem — all members achieved 3,500–3,850 accepted orders in the same window — but shifted the bottleneck to queue management. The queue grows at ~50–63 messages/second with a single worker, requiring 180 goroutines to keep pace with 60 orders/s demand.

3. **Lambda** eliminated operational complexity entirely. Cold start overhead was consistently 2.1–2.4% across all team members, cost was $0 at low volume, and scaling is fully managed by AWS. The trade-off is weaker delivery guarantees compared to SQS.

The right architecture depends on the business context: Lambda for simplicity and low cost at low volume, SNS/SQS/ECS for durability and control at scale.